ReFuGe: Feature Generation for Prediction Tasks on Relational Databases with LLM Agents (Online Appendix)

A Additional Analyses.

We provide additional analysis for our proposed method, ReFuGe.

A.1 Additional Results on RQ3.

RQ3. Effect of iteration with feedback. Does ReFuGe's performance improve through its feedback-driven iterative loop?

As shown in Figure 1, the evaluation performance of ReFuGe generally improves over iterations, demonstrating the effectiveness of its feedback-driven iterative loop. Iteration stops when no additional feature is selected, and ReFuGe performs an average of 2.4 iterations across all tasks.

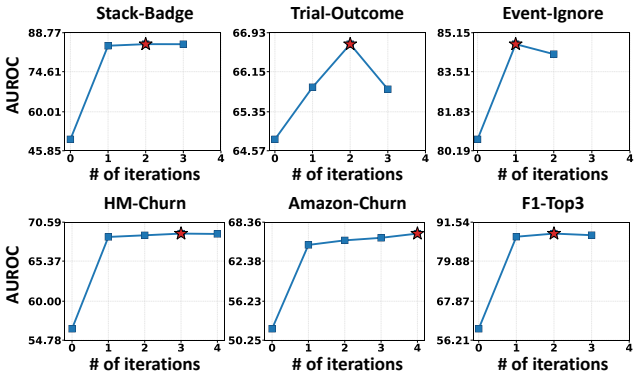

Figure 1: (RQ3) Evaluation performance tends to improve over iterations across all three tasks. The red ⋆ indicates the best validation performance over the iterations.

A.2 Additional Case Study on RQ4.

We provide an additional case study for the `rel-f1, driver-top3` task. As shown in Figure 2, ReFuGe generated reasonable and effective features to solve the prediction task.

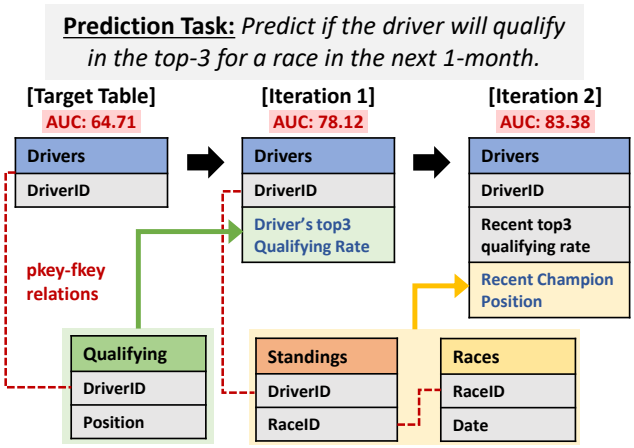

Figure 2: (RQ4) An example of feature generation by ReFuGe across iterations. ReFuGe iteratively generates and appends relational features to the target table, progressively enhancing prediction performance on RDBs.

A.3 Experimental Details

Dataset Details. To enhance understanding of prediction tasks in RDBs, we provide descriptions of the actual tasks below.

- **`Avito-visits`:** Predict whether each customer will visit more than one ad within the next 4 days.
- **`Avito-clicks`:** Predict whether each customer will click on more than one ad within the next 4 days.
- **`Stack-engage`:** Predict whether a user will make any votes, posts, or comments within the next 3 months.
- **`Stack-badge`:** Predict whether a user will receive at least one new badge within the next 3 months.
- **`F1-dnf`:** Predict whether a driver will fail to finish (DNF) a race within the next 1 month.
- **`F1-top3`:** Predict whether a driver will qualify in the top 3 for a race within the next 1 month.
- **`Event-repeat`:** Predict whether a user will attend an event (responding "yes" or "maybe") in the next 7 days, given that they have attended an event within the last 14 days.
- **`Event-ignore`:** Predict whether a user will ignore more than two event invitations within the next 7 days.
- **`Trial-outcome`:** Predict whether a clinical trial will achieve its primary outcome (defined as p-value < 0.05).
- **`HM-churn`:** Predict whether a customer will churn (i.e., make no transactions) within the next week.
- **`Amazon-churn`:** Predict whether a user will not review any product within the next 3 months.

Hyperparameter Settings. We employ Claude-Sonnet 4 and GPT 4.1 as backbone LLMs. In our method, two hyperparameters are involved: (1) the number of LLM instances used in the feature generation step (Step 2), and (2) the maximum number of iterations. For simplicity, we fix the number of LLM instances to three and the maximum number of iterations to four.

For the downstream tabular model, we tune only a small set of LightGBM hyperparameters: `num leaves`, `max depth`, and `feature fraction`. We search over the ranges: `num leaves` in $\{10, 31, 50\}$, `max depth` in $\{-1, 10, 15\}$, `feature fraction` in $\{0.8, 1.0\}$. Note that hyperparameter tuning is performed using the validation set.

B Prompt Details.

We provide the prompts used in each step of our framework (see Figures 3, 4, 5, and 6). Recall that our framework consists of three main steps: Step 1: Schema Selection, Step 2: Feature Generation, and Step 3: Feature Filtering. We additionally include the prompts used for generating executable code corresponding to the selected features.

Step 1. Schema Selection Prompt

You are in **STEP 1.1: Table Selection & Join Paths**.

Goal: Given the task description and target, select **helper tables** that would help solve the task and provide explicit join paths from TABLE: {entity_table} COL: {entity_col} to each helper table using PK/FK in the schema.

TASK: {task_description}

TARGET
- **target_table**: {entity_table}
- **target_key**: {entity_col}

Output Format
- TABLE 1
- TABLE 2
- TABLE 3

Now you are in **STEP 1.2: Column Selection**.

In the previous step, you choose the tables that would be helpful in the task.

Response in Step 1.1: {response from Step 1.1}

From these tables, please **select columns** that would be helpful for the task. We should generate new columns from aggregating these columns.

IMPORTANT
- Columns will be used for calculating new feature to augment {entity_table}.
- Reference columns that exist in the schema.
- Output a concise bulleted list of columns and tables**, each with a short rationale**.

Figure 3: Prompt of Step 1: Schema Selection.

Step 2. Feature Generation Prompt

You are now in **STEP 2: Feature Generation**.

Goal: Based on the responses from Step 1 (Tables and Columns), decide which aggregation functions to apply in order to generate up to three new features that augment the target table {entity_table}.

Response from Step 1.1: {response from step 1}
Response from Step 1.2: {response from step 2}

IMPORTANT
- Select up to three features to create.
- Each feature must be derived by applying a clear aggregation function (e.g., COUNT, SUM, AVG, MAX, MIN, STD).
- Each feature must be numeric or categorical (float or int).
- Each feature must join back to the target table on {entity_col}.
- Exactly one value per {entity_col}.
- Provide a short rationale for why this aggregation is useful for the task.
- Output a concise bulleted list of the final features.
- Avoid column-name collisions on joins.

Output Format (strict)
Feature 1:
- Name: feat{i}_1
- Plan:
- Rationale:

Feature 2:
- Name: feat{i}_2
- Plan:
- Rationale:

Feature 3:
- Name: feat{i}_3
- Plan:
- Rationale:

Figure 4: Prompt of Step 2: Feature Generation.

Step 3.1 Reasoning-based Feature Filtering Prompt

You are the **JUDGE LLM**. We have N=9 candidate feature plans (JSON objects), produced by 3 different LLMs (3 each).

Pick the **best 3** that are **(a) likely to be predictive for the task, (b) feasible to implement from the given schema/tables, and (c) not redundant with each other.** Favor central ideas (appear in multiple variants across models) while keeping diversity.

Return a JSON with:

```
 selected_ids: [id1, id2, id3],   the 3 chosen candidate ids (1..9)
 reasons: {{
    id1: <why chosen>,
    id2: <why chosen>,
    id3: <why chosen>
 }}
}}
```

Candidates (one per line): {features from step 2}

Figure 5: Prompt of Step 3: Reasoning-based Feature Filtering.

Execution (Coding) Prompt

Based on the three feature plans below, write a Python script **defining three separate functions** (compute_feature1, compute_feature2, compute_feature3).
Each should return a DataFrame with columns [entity_col, feat_name].

At the end, save each feature for train/val/test as a separate CSV file. Follow this format strictly (all three functions must be implemented, no narrative):

OUTPUT INSTRUCTIONS
1. Copy the entire skeleton exactly as shown, do NOT remove any line.
2. Replace each 'PLEASE FILL IN BELOW FUNCTION' section with working code.
3. The first line of your reply must be "```python" and the last line must be "```".
4. No narrative, no explanation — code only.
5. Each function should return a DataFrame with columns ONLY [entity_col,feat_name].
6. Do not use columns or tables that do not exist in the schema.
7. If there are any "timestamp" column type in a table**, prevent data leakage** by filtering rows using the appropriate cutoff (timemax_trn/val/tst) for the given split.
8. Ensure you produce exactly one value per {entity_col} (i.e. one row per entity).
9. When joining/merging two tables that may share column names, you MUST proactively disambiguate to avoid unintended overwrites.
   - In pandas: always pass an explicit `suffixes=('<left>','<right>')` to `pd.merge(...)`, or pre-rename columns before merging; never rely on ambiguous `_x/_y` silently.
   - Select only the necessary columns for the join/output (project columns explicitly) so the result has unique, **meaningful column names.**
10. After any join, drop/rename overlapping columns that are not needed so that the final returned DataFrame contains only [entity_col, feat_name]. and no duplicate or ambiguous names.

Final Selected Feature Plans (3)
{response from step 3.1}

---Code Skeleton---
{code_skeleton for execution}
---Done---

Figure 6: Prompt of Coding & Execution for generated features.